```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix, recall_s
```

```
data = pd.read_csv('/content/Telco_Customer_Churn.csv')
data.shape
```

```
(7043, 21)
```

```
data.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetSe
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fibe

5 rows × 21 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
data['TotalCharges'] = pd.to_numeric(data['TotalCharges'], errors='coerce')
original_rows = len(data)
data = data.dropna(subset=['TotalCharges'])
print(f"Total dropped rows with NaNs: {original_rows - len(data)}")
```

```
Total dropped rows with NaNs: 11
```

```
data.columns
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
target_feature = 'Churn'
numeric_features = ['MonthlyCharges', 'TotalCharges', 'tenure']
categorical_features = ['gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod']
```

```
X = data[numeric_features + categorical_features]
y = data[target_feature]
```

```
from collections import Counter
Counter(y)
#The data is imbalanced since yes and no are not 50-50
```

```
Counter({'No': 5163, 'Yes': 1869})
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.3, stratify=y)
#In train_test_split, the stratify parameter ensures that the proportions of class labels in the traini
```

```
y_train = y_train.map({'Yes': 1, 'No': 0})
y_test = y_test.map({'Yes': 1, 'No': 0})
y = y.map({'Yes': 1, 'No': 0})
```

```
print(f"Original data churn rate: {y.mean():.2f}")
print(f"Training data churn rate: {y_train.mean():.2f}")
print(f"Testing data churn rate: {y_test.mean():.2f}")
```

```
Original data churn rate: 0.27
Training data churn rate: 0.27
Testing data churn rate: 0.27
```

```
from sklearn.compose import ColumnTransformer
# Numeric Preprocessing Batch

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
# Categorical Preprocessing Batch

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
# Combine both batches with ColumnTransformer

preprocessor = ColumnTransformer(
    transformers = [
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)

    ],
    remainder = 'drop'
)

# Create the final, full-stack pipeline
clf_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(
        class_weight = 'balanced',
        random_state = 42,
```

```
    ))
])
```

```
clf_pipeline.fit(X_train, y_train)
```

```
▸ Pipeline
  ▸ preprocessor: ColumnTransformer
    ▸ num
      ▸ SimpleImputer
      ▸ StandardScaler
    ▸ cat
      ▸ SimpleImputer
      ▸ OneHotEncoder
  ▸ LogisticRegression
```

```
y_predict = clf_pipeline.predict(X_test)
print(y_predict)
```

```
[0 1 0 ... 0 0 0]
```

```
y_proba = clf_pipeline.predict_proba(X_test)[:, 1]
print(y_proba)
```

```
[0.04011402 0.53981169 0.37388594 ... 0.19654714 0.00867264 0.47606738]
```

```
print(recall_score(y_test, y_predict))
```

```
0.8288770053475936
```

```
print(confusion_matrix(y_test, y_predict))
```

```
[[1105  444]
 [  96  465]]
```

```
print(precision_score(y_test, y_predict))
```

```
0.5115511551155115
```

```
print(accuracy_score(y_test, y_predict))
```

```
0.7440758293838863
```

```
print(f1_score(y_test, y_predict))
```

```
0.6326530612244898
```

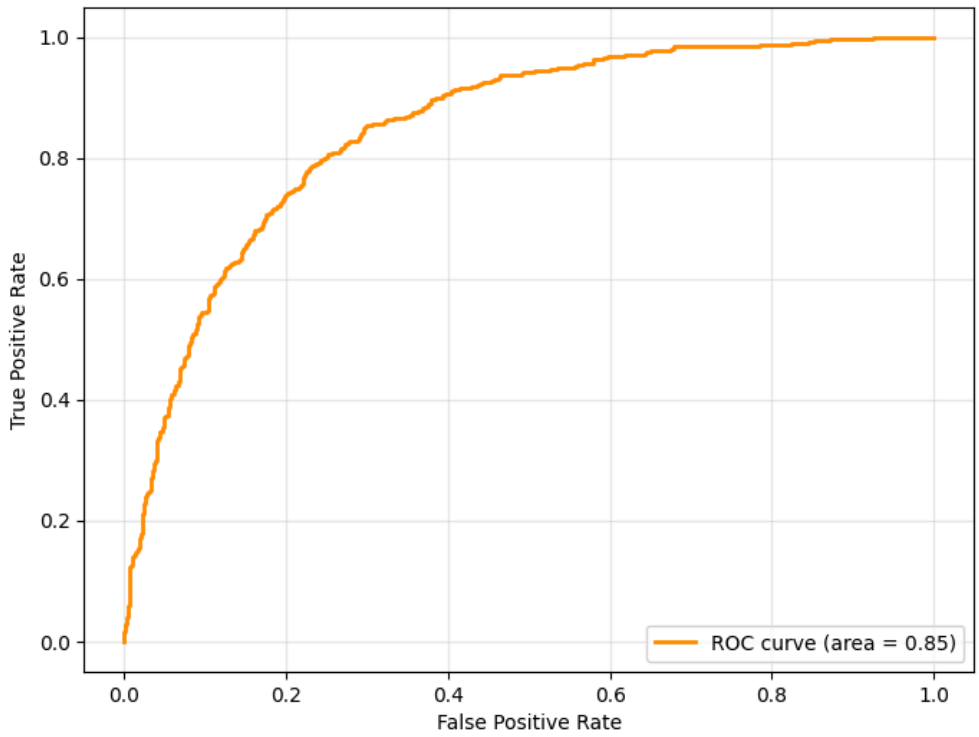
```
print(classification_report(y_test, y_predict))
```

```
              precision    recall  f1-score   support

           0       0.92      0.71      0.80      1549
           1       0.51      0.83      0.63       561

    accuracy                           0.74      2110
   macro avg       0.72      0.77      0.72      2110
weighted avg       0.81      0.74      0.76      2110
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)
print(roc_auc)
```

```
0.8491315770395254
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```




```
rf_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(
        n_estimators = 200,
        class_weight = 'balanced',
        random_state = 42,

    ))
])
```

```
rf_model.fit(X_train, y_train)
```

```
▸ Pipeline
  ▸ preprocessor: ColumnTransformer
    ▸ num
      ▸ SimpleImputer
      ▸ StandardScaler
    ▸ cat
      ▸ SimpleImputer
      ▸ OneHotEncoder
  ▸ RandomForestClassifier
```

```
y_pred_rf = rf_model.predict(X_test)
y_proba_rf = rf_model.predict_proba(X_test)[:, 1]
```

```
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.82      0.91      0.86      1549
           1       0.64      0.47      0.54       561
```

```
    accuracy                           0.79      2110
   macro avg       0.73      0.69      0.70      2110
weighted avg       0.78      0.79      0.78      2110
```

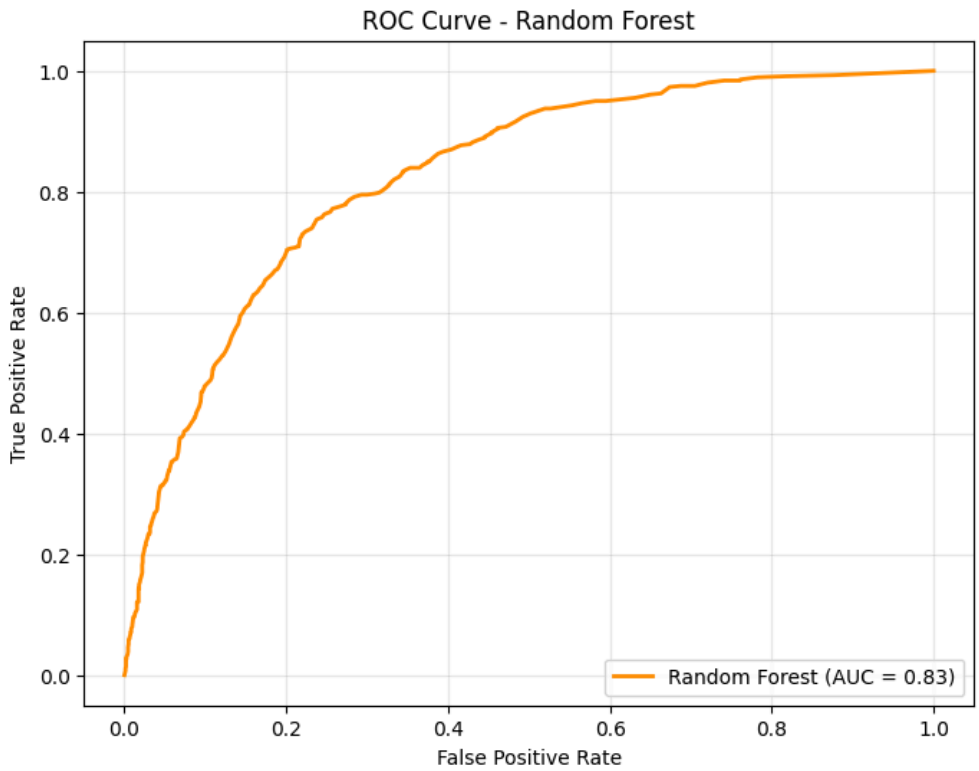
```
print(confusion_matrix(y_test, y_pred_rf))
```

```
[[1402  147]
 [ 299  262]]
```

```
fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_proba_rf)
roc_auc_rf = auc(fpr_rf, tpr_rf)
print(roc_auc_rf)
```

```
0.8251502608203325
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr_rf, tpr_rf, color='darkorange', lw=2, label=f"Random Forest (AUC = {roc_auc_rf:.2f})")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```




```
from sklearn.model_selection import cross_val_score

cv_auc = cross_val_score(
    clf_pipeline, X_train, y_train,
    scoring='roc_auc',
    cv=5
)
print("Logistic Regression CV AUC:", cv_auc.mean())
```

```
Logistic Regression CV AUC: 0.8416547102227959
```

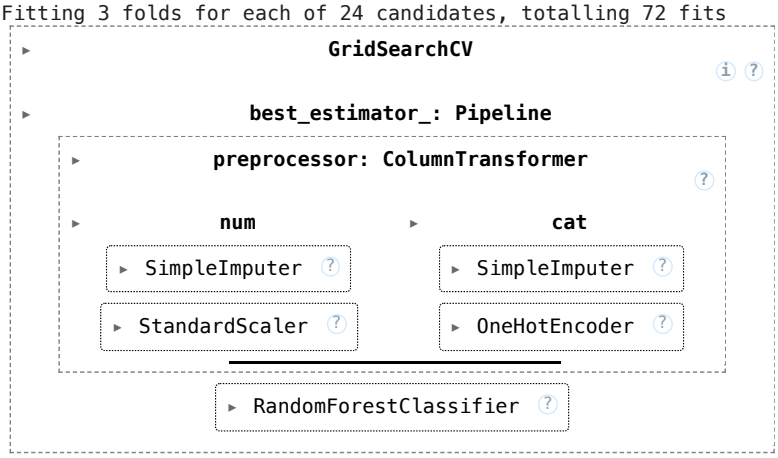
```
cv_auc = cross_val_score(
    rf_model, X_train, y_train,
    scoring='roc_auc',
    cv=5
)
print("Random Forest CV AUC:", cv_auc.mean())
```

```
Random Forest CV AUC: 0.8226561063990421
```

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'model__n_estimators': [200, 400],
    'model__max_depth': [None, 10, 20],
    'model__min_samples_split': [2, 10],
    'model__min_samples_leaf': [1, 5]
}
```

```
grid_rf = GridSearchCV(
    rf_model,
    param_grid,
    scoring='roc_auc',
    cv=3,
    n_jobs=-1,
    verbose=2
)

grid_rf.fit(X_train, y_train)
```

```
Fitting 3 folds for each of 24 candidates, totalling 72 fits
```




```
print("Best RF Params:", grid_rf.best_params_)
print("Best RF CV AUC:", grid_rf.best_score_)
```

```
Best RF Params: {'model__max_depth': 10, 'model__min_samples_leaf': 5, 'model__min_samples_split': 2, 'm
Best RF CV AUC: 0.8433915722531183
```

```
best_rf = grid_rf.best_estimator_

y_pred_rf = best_rf.predict(X_test)
y_pred_proba_rf = best_rf.predict_proba(X_test)[:, 1]

fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_pred_proba_rf)
roc_auc_rf = auc(fpr_rf, tpr_rf)
print(roc_auc_rf)
print(accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

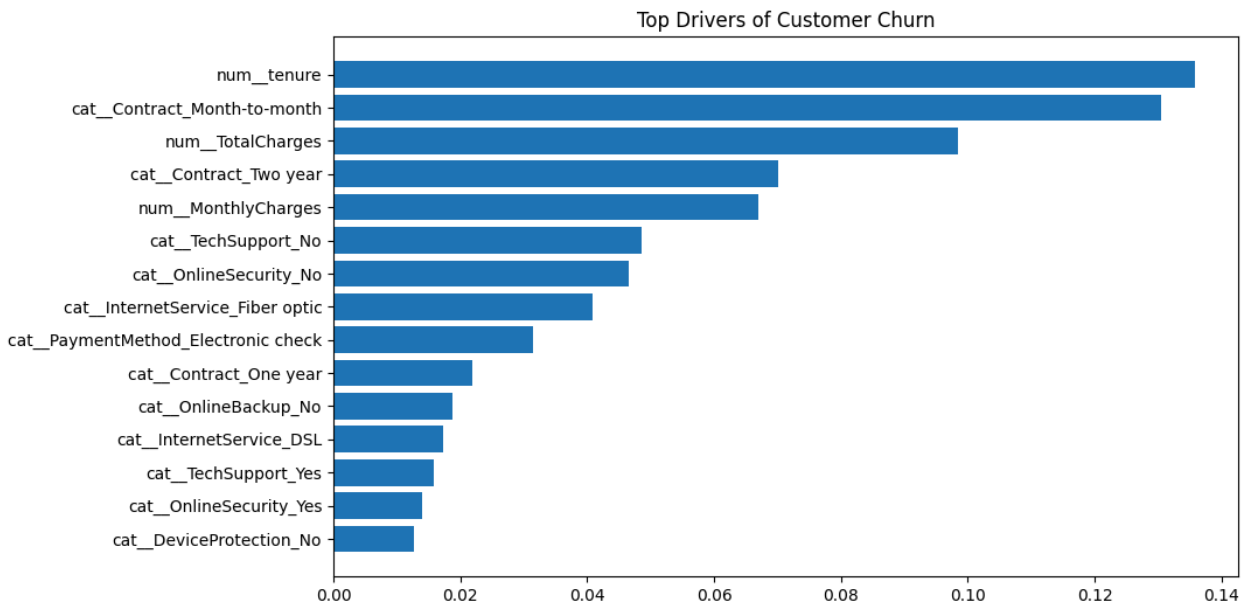
```
0.851225389504355
0.7805687203791469
              precision    recall  f1-score   support

           0       0.91      0.78      0.84      1549
           1       0.56      0.79      0.66       561

    accuracy                           0.78      2110
   macro avg       0.74      0.78      0.75      2110
weighted avg       0.82      0.78      0.79      2110
```

```
feature_names = best_rf.named_steps['preprocessor'].get_feature_names_out()
importances = best_rf.named_steps['model'].feature_importances_

feat_imp = pd.DataFrame({
    'feature': feature_names,
```

```
    'importance': importances
}).sort_values(by='importance', ascending=False)

feat_imp.head(15)
```

	feature	importance
2	num__tenure	0.135817
37	cat__Contract_Month-to-month	0.130520
1	num__TotalCharges	0.098363
39	cat__Contract_Two year	0.070185
0	num__MonthlyCharges	0.067065
28	cat__TechSupport_No	0.048515
19	cat__OnlineSecurity_No	0.046634
17	cat__InternetService_Fiber optic	0.040865
44	cat__PaymentMethod_Electronic check	0.031393
38	cat__Contract_One year	0.021945
22	cat__OnlineBackup_No	0.018837
16	cat__InternetService_DSL	0.017333
30	cat__TechSupport_Yes	0.015807
21	cat__OnlineSecurity_Yes	0.013949
25	cat__DeviceProtection_No	0.012741

```
top_features = feat_imp.head(15)
plt.figure(figsize=(10, 6))
plt.barh(top_features['feature'], top_features['importance'])
plt.gca().invert_yaxis()
plt.title("Top Drivers of Customer Churn")
plt.show()
```

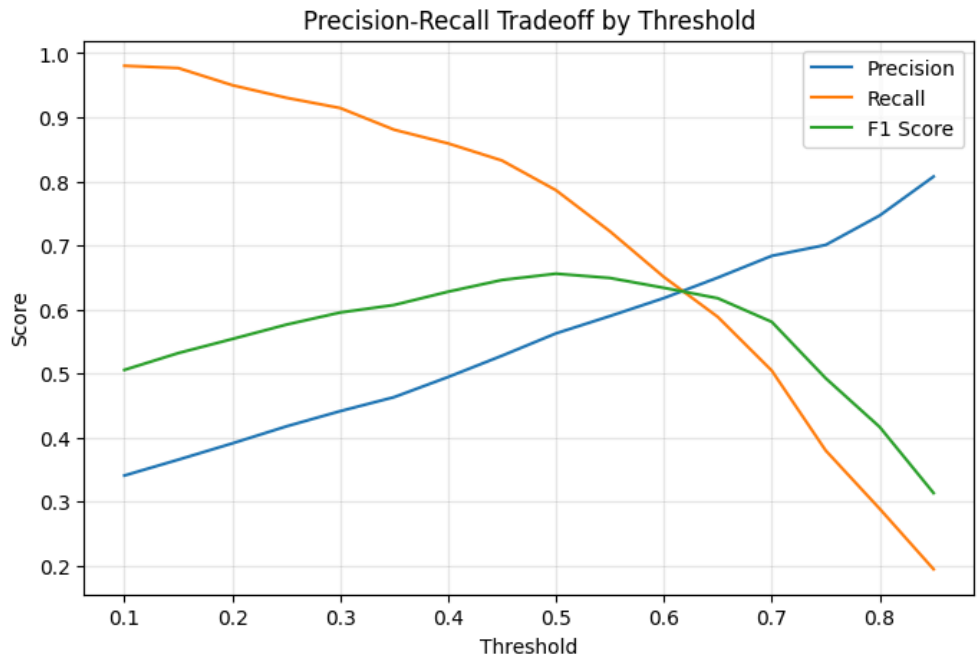



```
thresholds = np.arange(0.1, 0.9, 0.05)

results = []

for t in thresholds:
    y_pred_custom = (y_pred_proba_rf >= t).astype(int)

    precision = precision_score(y_test, y_pred_custom)
    recall = recall_score(y_test, y_pred_custom)
```

```
    f1 = f1_score(y_test, y_pred_custom)

    results.append((t, precision, recall, f1))

results_df = pd.DataFrame(results, columns=["threshold", "precision", "recall", "f1"])
results_df
```

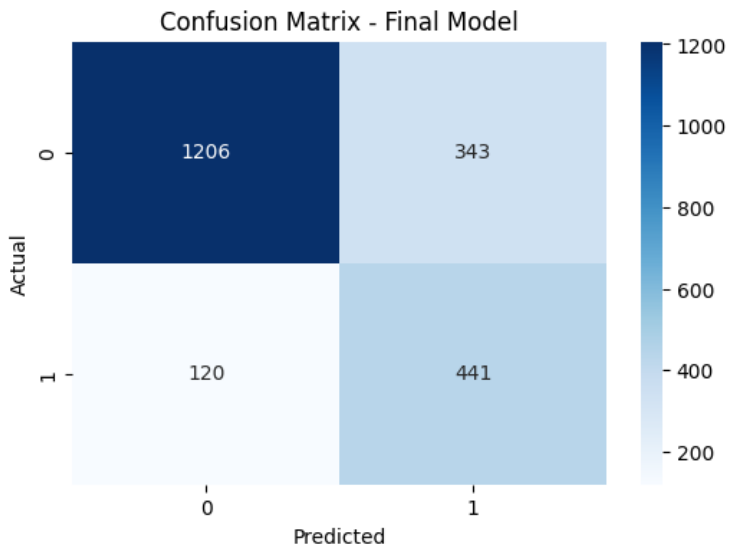
	threshold	precision	recall	f1
0	0.10	0.340557	0.980392	0.505515
1	0.15	0.365333	0.976827	0.531781
2	0.20	0.390762	0.950089	0.553766
3	0.25	0.417266	0.930481	0.576159
4	0.30	0.441101	0.914439	0.595128
5	0.35	0.462980	0.880570	0.606880
6	0.40	0.494359	0.859180	0.627604
7	0.45	0.527684	0.832442	0.645920
8	0.50	0.562500	0.786096	0.655762
9	0.55	0.589520	0.721925	0.649038
10	0.60	0.617597	0.650624	0.633681
11	0.65	0.649606	0.588235	0.617399
12	0.70	0.683575	0.504456	0.580513
13	0.75	0.700658	0.379679	0.492486
14	0.80	0.746544	0.288770	0.416452
15	0.85	0.807407	0.194296	0.313218

```
plt.figure(figsize=(8,5))
plt.plot(results_df["threshold"], results_df["precision"], label="Precision")
plt.plot(results_df["threshold"], results_df["recall"], label="Recall")
plt.plot(results_df["threshold"], results_df["f1"], label="F1 Score")
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.legend()
plt.title("Precision-Recall Tradeoff by Threshold")
plt.grid(alpha=0.3)
plt.show()
```

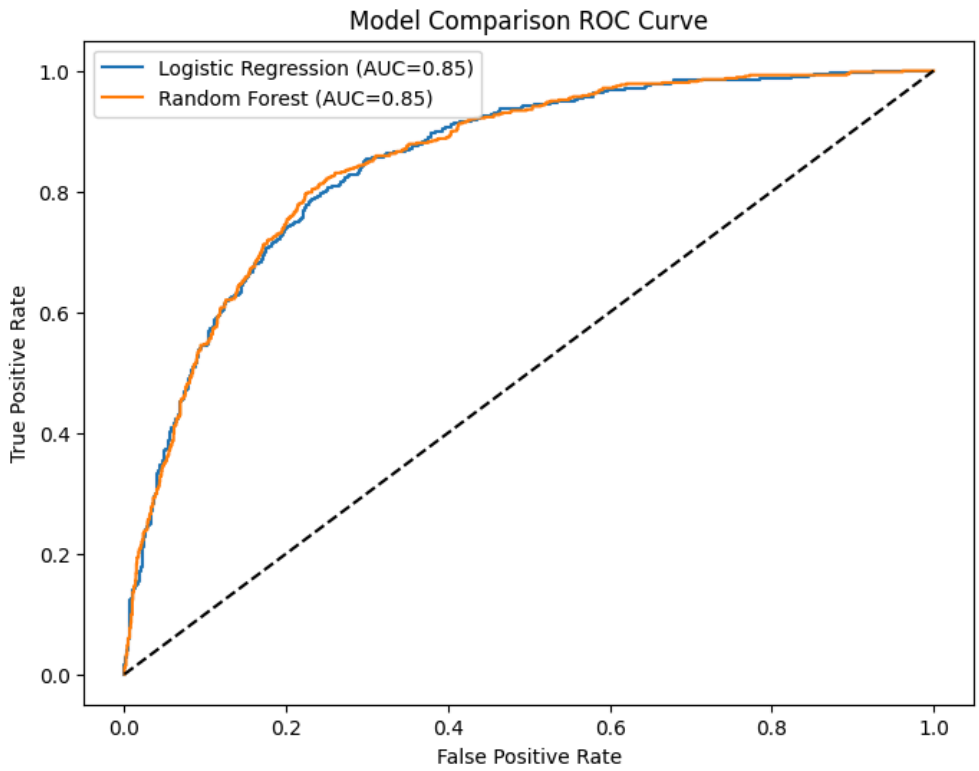



```
y_pred_final = best_rf.predict(X_test)

cm = confusion_matrix(y_test, y_pred_final)
```

```
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Final Model")
plt.show()
```




```
plt.figure(figsize=(8,6))

plt.plot(fpr, tpr, label=f"Logistic Regression (AUC={roc_auc:.2f})")
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest (AUC={roc_auc_rf:.2f})")

plt.plot([0,1],[0,1],'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Model Comparison ROC Curve")
plt.legend()
plt.show()
```




```
churn_prob = y_pred_proba_rf

risk_df = pd.DataFrame({
    "churn_probability": churn_prob
```

```
})

risk_df["risk_level"] = pd.cut(
    risk_df["churn_probability"],
    bins=[0,0.3,0.6,1],
    labels=["Low Risk","Medium Risk","High Risk"]
)

risk_df["risk_level"].value_counts()
```

risk_level	count
Low Risk	947
High Risk	591
Medium Risk	572

dtype: int64

Start coding or generate with AI.